

TECHNICAL OVERVIEW

repoggits

A home for student projects — software, hardware, and hybrid academic builds, from submission through peer review to public showcase.

TypeScript

Next.js 16

React 19

PostgreSQL / Neon

Three.js

Playwright

Tech Stack

Language

TypeScript across `app/` , `components/` , `lib/` , `scripts/` , `tests/` . Strict checking via `tsc --noEmit` .

Framework

Next.js 16 (App Router) with the Webpack build, React 19 / react-dom 19. A single catch-all route `app/api/[ ... path]/route.ts` handles all API endpoints.

Database

PostgreSQL — no ORM, raw SQL via `pg` . Built for Neon (managed serverless Postgres) but works with any Postgres instance.

Migrations

Idempotent SQL executed at startup inside an advisory-locked transaction — no separate migration tool/framework.

3D / Visuals

Three.js powers scene components (`CircuitScene` , `TeamScene` , `CodingScene`) for the interactive workbench.

Media & Files

`sharp` re-encodes images to WebP;
`yauzl` / `yazl` inspect and build ZIPs without executing contents.

Validation & Mail

`zod` schemas; email via `nodemailer` / `@azure/communication-email` , with an "outbox" mode that stores mail without sending.

Testing

Playwright drives browser E2E and API regression suites against isolated `repoggits_test_*` Postgres schemas.

Database Schema

All tables live under a configurable schema (`REPOGGITS_DB_SCHEMA` , default `repoggits`), enforced in `lib/db.ts` . File uploads are stored as raw bytes (`bytea`) directly inside Postgres — there is no external object storage (S3, etc.).

Table	Purpose
<code>users</code>	Accounts — student / teacher / superadmin roles, verification & suspension flags
<code>sessions</code>	Hashed session tokens with expiry
<code>tokens</code>	Email verify / reset / invite tokens, plus OTP code + attempt counter
<code>projects</code>	Project records — owner, featured/archived/example flags, views, downloads
<code>versions</code>	Versioned project content — status: draft / pending / approved / rejected / changes_requested
<code>reviews</code>	Reviewer decisions per version, with reason and audit trail
<code>audit</code>	System-wide action log
<code>notifications</code>	Per-user notification feed
<code>bookmarks</code> , <code>reactions</code>	Saved projects; star/like reactions
<code>comments</code>	Threaded discussion per project
<code>files</code>	Uploaded binary content (images, ZIP source), stored as <code>bytea</code>
<code>rate_limits</code>	Request throttling counters
<code>outbox</code>	Queued outgoing email
<code>settings</code>	Moderation rules and category taxonomy (departments, subjects, tags)

Project Structure

```
app/                                # Next.js pages (App Router) + the single catch-all API route
  page.tsx                          Homepage
  auth/                             Login / register / verify / reset
  submit/                           Project submission form
  workspace/                         Author's draft/version workspace
  account/                          User account settings
  projects/[id]/                     Public project detail page
  admin/, admin/settings/            Educator / Super Admin review & settings dashboards
  demo/                              Sample/demo page
  api/[...path]/                     All backend API endpoints (custom internal router)
components/
  platform/                          Project display, editor, workspace, admin, 3D scenes
  Workbench.tsx                      Top-level 3D workbench component
lib/                                 # Core server logic
  db.ts                              Postgres pool, schema/migration, query helpers
  auth.ts, api-auth.ts               Sessions, tokens, password hashing, request auth
  schema.ts                          Zod validation schemas
  policy.ts                          Authorization / moderation policy
  projects.ts, project-display.ts, project-community.ts
  api-files.ts, file-validation.ts    Upload handling & validation
  mail.ts                            Email sending (SMTP / Azure / outbox)
  draft-recovery.ts                  Browser-side draft persistence support
  site-backup.ts                     Admin source-code backup ZIP generator
  errors.ts, http.ts                 Shared error types & HTTP helpers
scripts/                             # CLI tools run via tsx, outside the Next.js server
  setup.ts                           Runs migrations, bootstraps first Super Admin
  seed-sample.ts                      Seeds the "CampusFlow" example project
  outbox.ts, send-outbox.ts           Inspect / send queued emails
examples/campusflow/                 Runnable sample student project (source + assets)
public/                              Fonts, images, sample demo files
tests/                               Playwright E2E, dev-mode, and OTP test suites
deploy/                              VPS installer, systemd unit, nginx/Caddy configs
docs/                                Deployment & operations guides, screenshots
Dockerfile, docker-compose.yml       # Single app container; DB is external (e.g. Neon)
```

Architecture Notes

- ▶ **No ORM** — all database access is raw parameterized SQL through a thin wrapper (`db.query` , `transaction`) in `lib/db.ts` .
- ▶ **No external object storage** — uploaded files are stored directly as `bytea` in Postgres.
- ▶ **No queue/job system** — email uses a simple `outbox` table, drained via script or sent inline through SMTP/Azure.
- ▶ **Single Node process** deployment target, sized for a 1 vCPU / 2 GB VPS.
- ▶ **Custom session & auth** — hashed session/token rows in Postgres, no third-party auth provider. Roles: student, teacher, superadmin.

- ▶ **Review workflow** — versions move `draft` → `pending` → `approved` / `rejected` / `changes_requested`, with a configurable 1- or 2-reviewer approval threshold and full audit logging.
-

Deployment: Docker Compose runs the app container (port 3000) with an optional Caddy TLS profile — the database is expected to be external (e.g. Neon). A non-Docker path installs Node 24, PostgreSQL, and nginx directly via `deploy/install-vps.sh`. Full details in `docs/DEPLOYMENT.md` and `docs/OPERATIONS.md`.